

Business Information Systems

Weeks 03 & 04 - Internal & External Integration · Business Intelligence (BI) and EIS



Cyber.SoHo Educational Hub

From the Industry to the Classroom — Building the IT's Future, Today and Together!

2-hour lecture · Undergraduate



Two lessons today — and both are named on purpose

1

Lesson 1

Internal and External Integration

How the data gets to one trustworthy place. (Hour 1)

2

Lesson 2

Business Intelligence and EIS

How that data becomes a decision. (Hour 2)

Every example follows the same four-beat pattern — so nothing feels scary:

1

The task

what we're doing

2

A real story

Northern Roast

3

A picture

diagram / chart

4

Runnable code

you can run it



What you'll actually be able to do

- Explain what it means for business systems to be integrated — and why a company falls apart when they aren't.
- Tell internal integration (inside one company) from external integration (across company borders).
- Recognise the three wiring patterns: point-to-point, hub-and-spoke, and the ESB.
- Describe BI (Business Intelligence) and trace data from a cash register to an executive's dashboard.
- Separate a database's two jobs: OLTP (live transactions) vs OLAP (historical analysis).
- Place every system in the pyramid: TPS → MIS → DSS → EIS.
- Read a small dataset and pull a real decision out of it — like an analyst on day one.



The big idea

Integration is the plumbing. BI is the tap you actually drink from. A healthy business needs both — or it either drowns in disconnected systems or starves for insight.



1

LESSON 1

Internal and External Integration

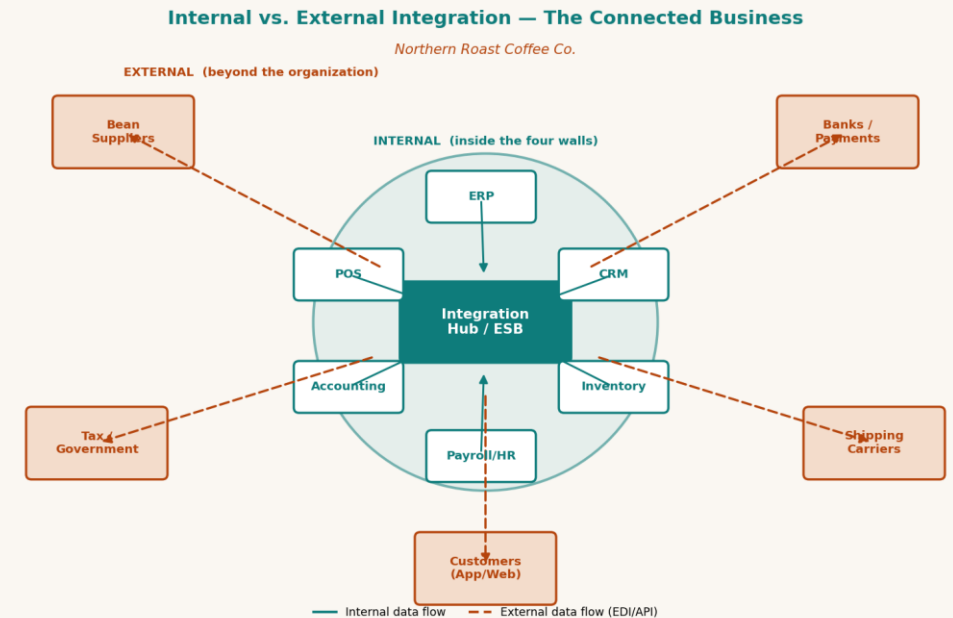
One company, lots of little systems, all needing to share the same truth. This hour is about getting them to talk.



LESSON 1 · POINT 1.1

Data silos: the same fact, stored three different ways

- A silo is a system that hoards its own copy of data and doesn't share it.
- When Marketing, E-commerce, and Finance each keep their own customer list, the same person ends up with three slightly different records.
- Nobody is lying — the copies just drifted. A new email here, an old address there.
- Result: reports disagree, customers get annoyed, and staff waste time re-typing.
- Integration exists to replace many drifting copies with one shared, trusted truth.





EXAMPLE · POINT 1.1

One customer, three conflicting records

Task: compare how three departments store the same loyalty member, Maya, and flag every disagreement — the exact problem integration solves.

System	Name	Email	City
Marketing	Maya Tremblay	maya.t@email.com	Plateau
E-commerce	Maya Tremblay	maya.t@email.com	Mile-End
Finance	M. Tremblay	maya.tremblay@oldjob.com	Plateau

- Three systems, three "truths." Which city does Maya live in? Which email gets the receipt? Nobody can say — and that is a data silo in one picture.



Code: a tiny silo-conflict detector

What it does: holds each department's version of Maya, then checks every field for disagreement.

```
python

# Same person, three departments, three records:
marketing = {"name": "Maya Tremblay", "city": "Plateau"}
ecommerce = {"name": "Maya Tremblay", "city": "Mile-End"}
finance    = {"name": "M. Tremblay", "city": "Plateau"}
silos = {"Marketing": marketing,
         "E-commerce": ecommerce, "Finance": finance}

# For each field, gather answers and flag conflicts:
for field in ["name", "city"]:
    vals = {t: r[field] for t, r in silos.items()}
    if len(set(vals.values())) > 1:
        print("CONFLICT on", field, "->", vals)
```

OUTPUT

```
CONFLICT on name ->
    Maya Tremblay vs M. Tremblay
CONFLICT on city ->
    Plateau vs Mile-End
```

In plain English

We line up the three copies, then for each field turn the answers into a set. One unique value = agreement; more than one = a flagged conflict. The silo problem, visible in ten lines.



Internal integration: systems talking inside one company

- **Internal integration** wires together the systems one company owns so a single event updates everything at once.
- **ERP (Enterprise Resource Planning)** is one suite for finance, inventory, HR — all on a single database.
- **CRM (Customer Relationship Management)** tracks customers and their history (e.g. loyalty).
- The dream: one sale is typed once, and inventory, accounting, and the loyalty profile all update by themselves — no re-typing, no drift.

One sale → many updates

Sell a latte once. Internally that single event lowers inventory, books revenue, and adds loyalty points — automatically.

Try it yourself

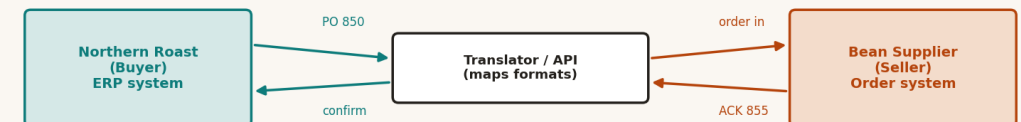
The downloadable `internal_integration_demo.html` shows one shared 'truth' several panels read from — click and watch them move together.



External integration: talking across company borders

- **External integration** connects your systems to other organizations — suppliers, banks, the tax authority.
- **EDI (Electronic Data Interchange)** — a long-standing standard for exchanging documents (orders, invoices) computer-to-computer.
- **API (Application Programming Interface)** — a defined doorway letting one program request data or actions from another in real time.
- Example: when beans run low, Northern Roast's system fires an order straight to the supplier — no phone call, no human, no typo.

Automated Purchase Order — Retailer ⇌ Supplier (EDI / API)



1. Stock hits reorder point
2. PO created automatically
3. Sent as standard message
4. Supplier confirms & ships
5. Invoice flows back (810)



Code: an automatic purchase-order generator

What it does: scans inventory and, for anything at/under its reorder point, builds an order line for a supplier.

 `python`

```
# Each item knows its stock, trigger, order qty:
inventory = [
    {"sku": "BEAN-HOUSE", "stock": 9, "reorder_point": 12, "qty": 50},
    {"sku": "CUP-12OZ", "stock": 4, "reorder_point": 8, "qty": 100},
]
purchase_order = []
# If stock has hit the trigger, add an order line:
for item in inventory:
    if item["stock"] <= item["reorder_point"]:
        purchase_order.append(
            {"sku": item["sku"], "qty": item["qty"]})
# This PO crosses the border via EDI / API.
```

OUTPUT

```
PURCHASE ORDER ->
Bean & Cup Supply Co.
BEAN-HOUSE      qty: 50
CUP-12OZ        qty: 100
```

In plain English

We walk the inventory once. Each low item produces a clean order line. The finished list is exactly the message that leaves the company and arrives at the supplier — external integration in action.



Three ways to wire systems together

Point-to-Point

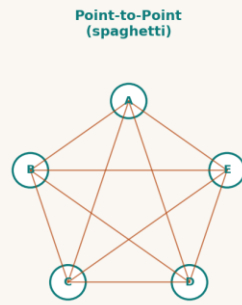
Wire each system directly to every other. Simple for 2–3 systems; a tangled nightmare as you grow.

Hub-and-Spoke

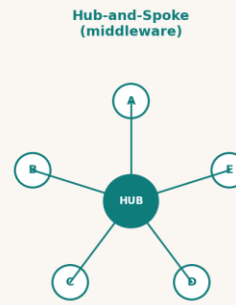
One central piece of middleware sits in the middle; every system connects only to the hub. Far fewer cables.

ESB (Enterprise Service Bus)

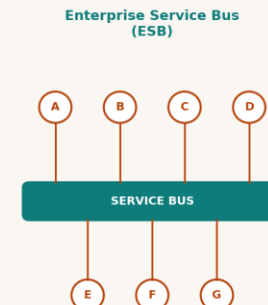
A shared 'bus' systems plug into that can route, transform, and monitor messages. The idea behind SOA.



10 links for 5 apps — grows fast!



5 links — one central broker



Apps plug into a shared 'bus'



EXAMPLE · POINT 1.4

Why hub-and-spoke wins: it's just arithmetic

Task: count the connections Northern Roast needs as it grows — point-to-point vs hub-and-spoke.

Point-to-point

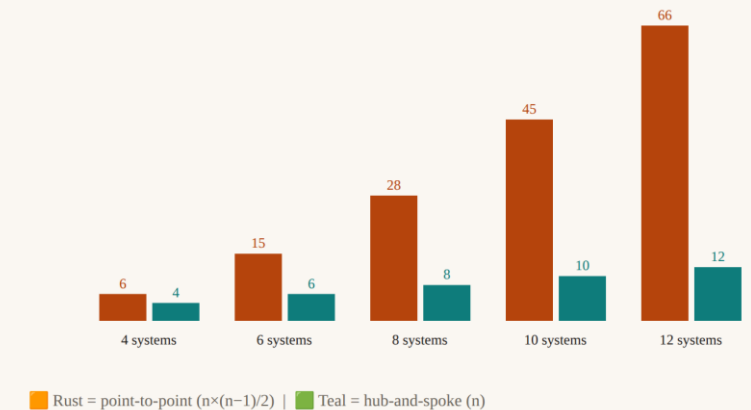
$$n \times (n - 1) \div 2$$

Hub-and-spoke

$$n$$

Systems (n)	Point-to-point	Hub-and-spoke
5	10 links	5 links
8	28 links	8 links
12	66 links	12 links

Connections needed as Northern Roast grows





Integration: the upside, the risk, the safety net

BENEFITS

- One shared truth
- Less re-typing, fewer errors
- Faster reporting
- Automation across borders

RISKS

- Tight coupling = fragile
- A bad record spreads everywhere
- Security: more doors to guard
- One outage can ripple

Safety net: a data contract

Agree up front on the exact fields and types each side will send, then validate every incoming message and reject anything that breaks the contract — before bad data spreads. (See OWASP for doing this securely.)



Code: a data-contract validator

What it does: checks each incoming message against agreed field names and types; accepts only clean data.

python

```
# The contract: required field -> required type
CONTRACT = {"order_id":str, "sku":str,
            "quantity":int, "unit_price":float}

def validate(message):
    problems = []
    for field, want in CONTRACT.items():
        if field not in message:
            problems.append("missing "+field)
        elif not isinstance(message[field], want):
            problems.append("bad type "+field)
    return (len(problems) == 0, problems)
```

OUTPUT

```
GOOD: ACCEPTED
BAD : REJECTED ->
    missing quantity;
    bad type unit_price
```

In plain English

For every required field we ask two questions: is it here, and is it the right type? Zero problems = ACCEPTED; otherwise reject and say why. This gate stops one bad record from poisoning every integrated system.



Watch this for Lesson 1 (≈ 6 minutes)



What is Software Integration? Enterprise Integrations Explained in 6 Minutes

youtube.com/watch?v=8MSBTaag7Wo

- Why this one: a clear, beginner-friendly tour of why systems integrate and how — a perfect 6-minute primer for Lesson 1.

Copyright & usage note

Referenced by link only — not embedded, copied, downloaded, or re-hosted. All rights remain with the creator and YouTube. Linking is consistent with YouTube's Terms of Service; use here is educational and non-commercial. Cyber.SoHo claims no ownership, affiliation, or endorsement. Confirm the link still plays before class.



Internal & External Integration

- A silo hoards its own copy of data; integration replaces many drifting copies with one shared truth (1.1).
- Internal integration (ERP, CRM) makes one event update every system inside the company (1.2).
- External integration (EDI, API) lets systems talk across company borders (1.3).
- Three patterns — point-to-point, hub-and-spoke, ESB — and the maths says skip point-to-point at scale (1.4).
- Integration is powerful but coupling is risky; a data contract validates and rejects bad data at the door (1.5).



2

LESSON 2

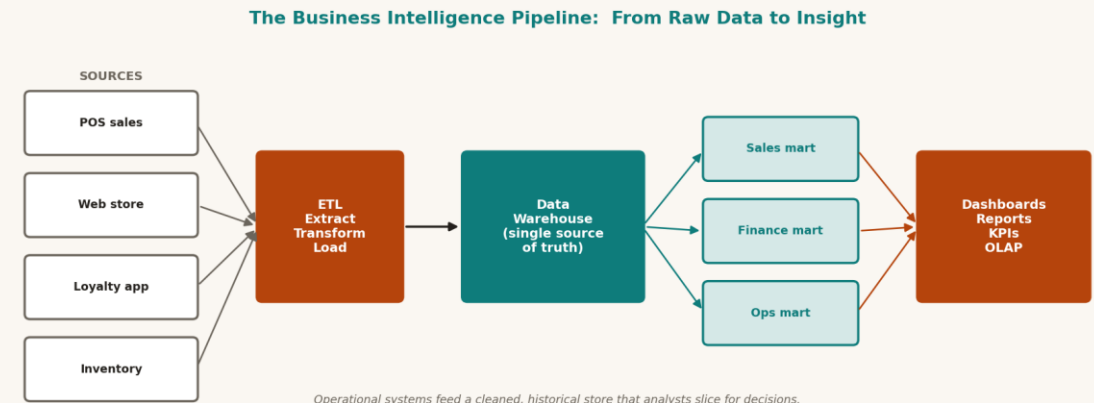
Business Intelligence and EIS

Messy raw data flows in one end; a clean decision comes out the other. This hour is the pipeline.



From raw data to a decision: the BI pipeline

- **BI (Business Intelligence)** turns collected data into evidence you can act on.
- The journey: raw events are extracted, cleaned (ETL), stored in a data warehouse, sliced by OLAP, and shown on a dashboard.
- Each stage adds trust. Skip the cleanup and every later number is suspect.
- We build every stage by hand — over Northern Roast's real 14-day sales file — and end at an executive's screen.





BI = evidence over gut feeling

- Descriptive analytics answers 'what happened?' — and even a few honest lines over a real file counts.
- A gut feeling says 'Westmount feels busy.' BI says 'Downtown earned \$100,776; Westmount \$46,432.'
- Numbers don't end arguments by being loud — they end them by being checkable.
- The skill that gets you hired on day one: read a dataset and pull one real decision out of it.

#

The 14-day file

4 stores × 14 days of sales. Everything in Lesson 2 — ranking, ETL, OLAP, dashboard, what-if — is computed from this single CSV you can download and re-run yourself.



The result: ranking four stores by revenue

Task: total 14 days of sales per store and rank them — the simplest, most useful BI there is.

Total revenue (14 days)

\$276,639

Transactions

37,482

Avg transaction (ATV)

\$7.38

Rank	Store	Revenue	Transactions
1	Downtown	\$100,776	13,531
2	Mile-End	\$67,737	9,221
3	Plateau	\$61,695	8,206
4	Westmount	\$46,432	6,524

Downtown earns more than twice Westmount — the kind of fact that starts a real business conversation.



Code: ranking stores from the CSV

```
python
import csv
from collections import defaultdict
revenue_by_store = defaultdict(float)

# Add each row's revenue to its store's total:
with open("data/northern_roast_daily_sales.csv") as f:
    for row in csv.DictReader(f):
        revenue_by_store[row["store"]] \
            += float(row["revenue_cad"])

# Sort biggest-first so the winner is on top:
ranking = sorted(revenue_by_store.items(),
                  key=lambda p: p[1], reverse=True)
```

OUTPUT

```
1. Downtown    $100,776
2. Mile-End    $67,737
3. Plateau     $61,695
4. Westmount   $46,432
```

In plain English

A defaultdict lets us add to each store's total without setup. We read the file row by row, sum revenue per store, then sort. Twelve lines turn a raw CSV into a ranked decision — descriptive BI.



Data warehouse, ETL, and data marts

- **Data warehouse (DW)** — a central store of cleaned, historical data built for analysis, not daily sales.
- **ETL (Extract, Transform, Load)** — the pipeline that pulls raw data, cleans it, and loads it into the warehouse.
- **Data marts** — department-sized slices of the warehouse (Finance's mart, Marketing's mart).
- It's the unglamorous cleanup — but it's why every later number can be trusted. Tools: Power Query, Talend.



Garbage in, garbage out

If 'downtown', 'DOWNTOWN', and ' Plateau ' aren't standardised, your tidy ranking is quietly wrong. ETL is where you earn trust.



OLTP vs OLAP next

Once data is clean and warehoused, we can ask big historical questions without slowing the live tills — Point 2.3.



Code: a mini-ETL that cleans messy rows

```
python
# Raw export: mixed case, text price, a duplicate:
raw_rows = [
    {"store": "downtown", "revenue": "6 547.17"},
    {"store": "DOWNTOWN", "revenue": "6 547.17"}, # dup
    {"store": " plateau ", "revenue": "3537.90"},
]
warehouse, seen = [], set()
for row in raw_rows:
    store = row["store"].strip().title() # tidy
    rev = float(row["revenue"].replace(" ", "")) # ->num
    if store not in seen: # de-dup
        seen.add(store)
        warehouse.append({"store": store, "revenue": rev})
```

OUTPUT

```
RAW:  downtown /
      DOWNTOWN / ' plateau '
CLEAN:
      Downtown 6547.17
      Plateau  3537.90
```

In plain English

Transform = strip spaces, fix casing, turn text into numbers.
Load = append the clean row, skipping anything seen before.
Messy in, trustworthy out.



OLTP vs OLAP: two jobs a database can have

OLTP — the till

- Online Transaction Processing
- Fast, tiny, live writes
- 'Record this sale, now'

OLAP — the cube

- Online Analytical Processing
- Big, sliced, historical reads
- 'How do weekends compare?'

OLTP vs. OLAP — Two Jobs, Two Databases

OLTP

Online Transaction Processing

- Runs the business NOW
- Many tiny read/writes
- 'Charge this card'
- Current data
- Optimized for speed/writes
- Example: the POS till

OLAP

Online Analytical Processing

- Understands the business
- Few huge read queries
- 'Sales by region, 3 yrs'
- Historical data
- Optimized for analysis
- Example: the BI dashboard



Code: an OLAP weekday vs weekend cube

python

```
import csv
from datetime import datetime
from collections import defaultdict
cube = defaultdict(lambda: {"weekday":0.0,"weekend":0.0})

with open("data/northern_roast_daily_sales.csv") as f:
    for row in csv.DictReader(f):
        d = datetime.strptime(row["date"],"%Y-%m-%d")
        b = "weekend" if d.weekday()>=5 else "weekday"
        cube[row["store"]][b] += float(row["revenue_cad"])
# Each store now has weekday vs weekend totals.
```

OUTPUT

Store	Wkday	Wkend	%
Downtown	67,157	33,619	33%
Mile-End	43,832	23,905	35%
Plateau	41,106	20,589	33%
Westmount	30,816	15,615	34%

In plain English

We bucket each day as weekday or weekend, then sum revenue into the right cell per store. The result is a tiny cross-tab — the 'face' of an OLAP cube — answering a question no single sale could.



Dashboards & KPIs: the human-facing tip of BI

A **KPI (Key Performance Indicator)** is a number chosen with a target because it measures something that matters. Five we track:

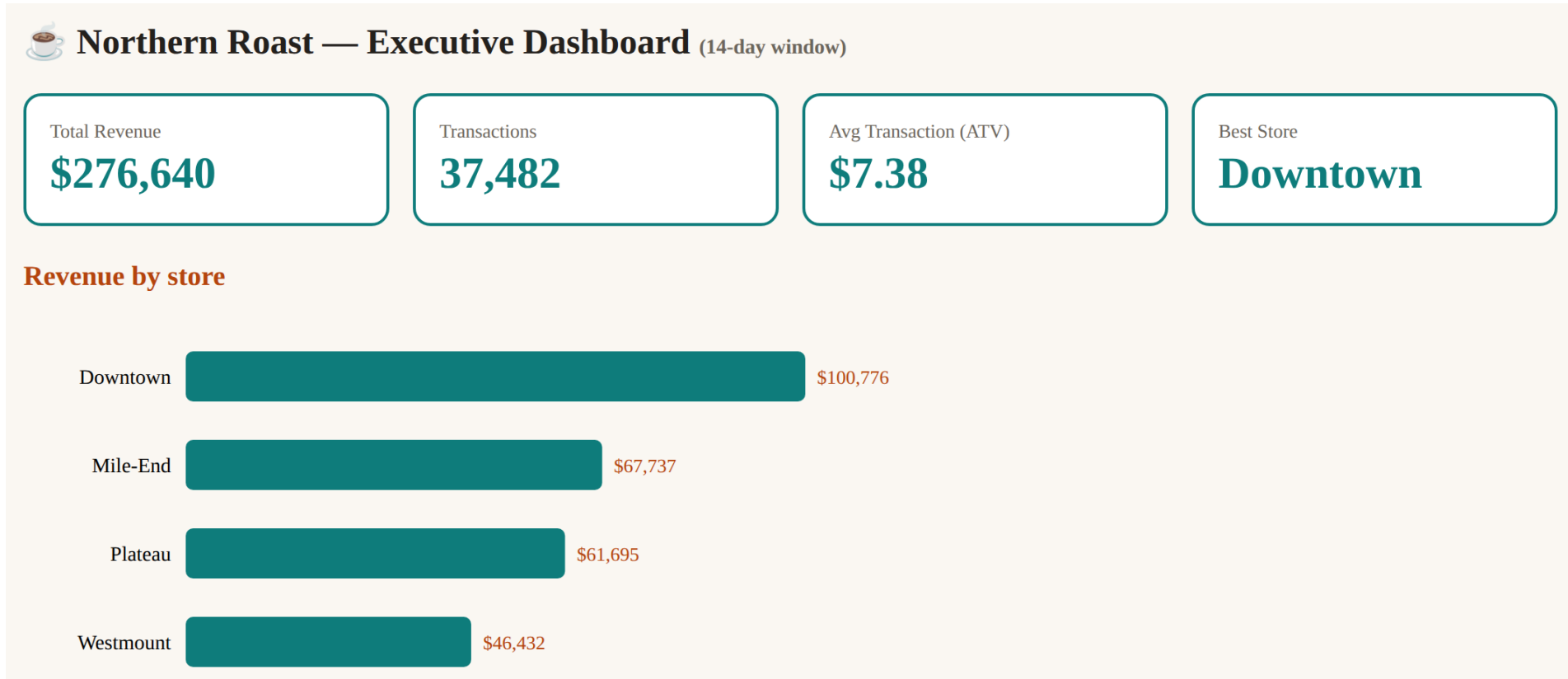
KPI	Plain meaning	Formula	Why it matters
Total Revenue	All money taken in	$\text{sum}(\text{revenue})$	Top-line health
Avg Transaction (ATV)	Spend per sale	rev / txns	Buying more?
Units per Txn (UPT)	Items per sale	$\text{units} / \text{txns}$	Up-selling?
Revenue per Store	Money each site makes	rev by store	Who performs?
Day-over-Day Growth	Change vs yesterday	$(t-y)/y$	Momentum?

- A dashboard turns these into cards and charts an executive reads in five seconds — which is exactly what we built (next slide).



The showpiece: a real executive dashboard

Task: put the KPIs and the store ranking on one screen — built by hand, no libraries (`executive_dashboard.html`).

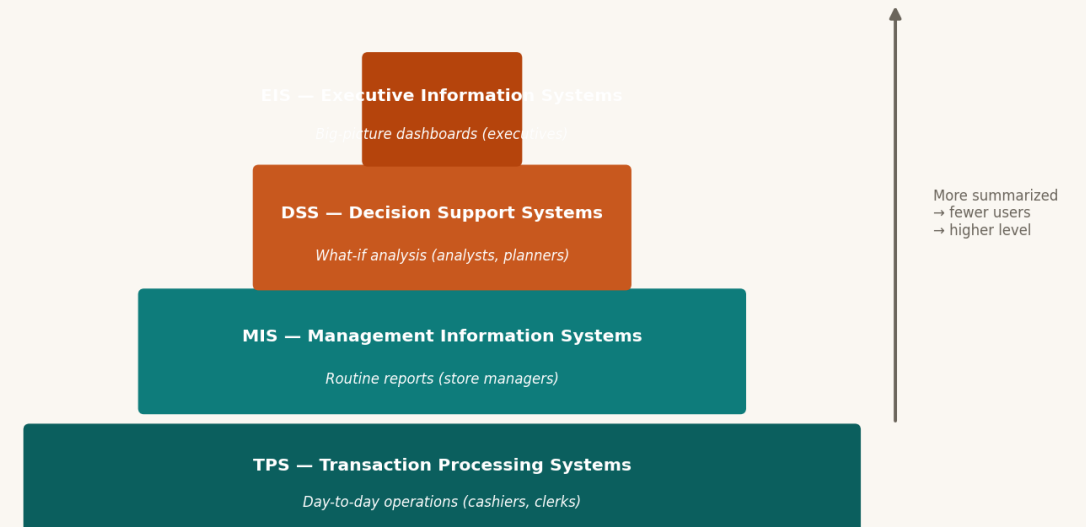




The information systems pyramid

- **TPS** (Transaction Processing System) — records day-to-day events, e.g. a sale.
- **MIS** (Management Information System) — routine summary reports for managers.
- **DSS** (Decision Support System) — models 'what if?' to help leaders choose.
- **EIS** (Executive Information System) — the high-level, org-wide dashboard at the top.
- Climb the pyramid: data gets more summarised, the audience more senior.

The Information Systems Pyramid — Who Uses What





Code: a DSS 'what-if' for leadership

What it does: models trimming the weakest store and promoting the flagship, then reports the net effect.

```
python

current = {"Downtown":100776,"Mile-End":67737,
           "Plateau":61695,"Westmount":46432}
# Leadership's tunable levers (assumptions):
westmount_loss = 0.06 # cut quiet hours -> -6%
downtown_lift = 0.08 # weekend promo -> +8%
scenario = dict(current)
scenario["Westmount"] *= (1 - westmount_loss)
scenario["Downtown"] *= (1 + downtown_lift)
delta = sum(scenario.values()) - sum(current.values())
print("Net change:", round(delta))
```

OUTPUT

```
Baseline : $276,640
Scenario : $281,916
Net      : +$5,276
          (BETTER OFF)
```

In plain English

Change the levers and the answer changes — that's what makes it a DSS. On these assumptions the chain ends ~\$5,276 ahead, tested before spending a dollar. That forward-looking model is the top of the pyramid.



Watch this for Lesson 2 (≈ 8 minutes)



What is Business Intelligence (BI) and Why is it Important? (Updated for 2024)

youtube.com/watch?v=uUTMiUuFJdA

- Why this one: a modern, plain-language case for why data-backed decisions beat gut instinct — the heart of Lesson 2.

Copyright & usage note

Referenced by link only — not embedded, copied, downloaded, or re-hosted. All rights remain with the creator and YouTube. Linking is consistent with YouTube's Terms of Service; use here is educational and non-commercial. Cyber.SoHo claims no ownership, affiliation, or endorsement. Confirm the link still plays before class.



Business Intelligence and EIS

- BI turns data into evidence; even a few honest lines over a real file is descriptive analytics (2.1).
- ETL feeds a data warehouse, sliced into department data marts — the cleanup that makes everything trustworthy (2.2).
- OLTP runs the business now; OLAP understands it over time. Keep them apart (2.3).
- A dashboard of KPIs and charts is BI's human-facing tip — and you built a real one (2.4).
- The pyramid runs TPS → MIS → DSS → EIS; higher = more summarised, more senior (2.5).



The whole two hours in one journey

One customer, one tap at 9:14 a.m. — follow it from the till all the way to the boardroom.

1 Tap

A latte is ordered & paid

2 Integrate

One sale updates inventory, books, loyalty; a low-bean order crosses to the supplier

3 ETL + DW

Overnight the sale is cleaned and warehoused

4 OLAP

Sliced by store and weekday

5 Dashboard / EIS

KPIs surface for leadership

6 DSS

A 'what if we renovate?' decision is modelled

That round trip — from one tap to a boardroom decision — IS Business Information Systems.



A 60-second self-check

Question (answer in your own words)	Where to look
Why does re-typing one sale into three systems cause trouble?	Point 1.1
What's the difference between internal and external integration?	Points 1.2 & 1.3
Why is hub-and-spoke usually calmer than point-to-point?	Point 1.4
What does ETL actually do to raw data?	Point 2.2
Why keep OLTP and OLAP apart?	Point 2.3
What turns a number into a KPI?	Point 2.4
Where does an executive's 'what if?' get answered?	Point 2.5

Can you answer all seven? You're ready. If not, the Point number tells you exactly where to re-read.



Glossary of abbreviations (1 / 2)

Abbr.	Full term	Plain meaning
API	Application Programming Interface	A doorway one program uses to call another in real time.
ATV	Average Transaction Value	Revenue / transactions — typical spend per visit.
BI	Business Intelligence	Turning data into decisions you can act on.
BIS	Business Information Systems	How organizations use software and data to run and decide.
CRM	Customer Relationship Management	Software tracking customers and their history.
DSS	Decision Support System	Models 'what if?' scenarios to help leaders choose.
DW	Data Warehouse	Central store of cleaned, historical data for analysis.
EDI	Electronic Data Interchange	Standard for exchanging documents computer-to-computer.
EIS	Executive Information System	High-level, org-wide dashboard for senior leaders.
ERP	Enterprise Resource Planning	One suite (finance, inventory, HR) on a shared database.
ESB	Enterprise Service Bus	A central messaging spine systems plug into.
ETL	Extract, Transform, Load	Pipeline that pulls, cleans, and loads data.



Glossary of abbreviations (2 / 2)

Abbr.	Full term	Plain meaning
KPI	Key Performance Indicator	A number chosen with a target because it matters.
MIS	Management Information System	Routine summary reports for middle managers.
OLAP	Online Analytical Processing	Big, sliced, historical queries ('the cube').
OLTP	Online Transaction Processing	Fast, small, live transactions ('the till').
P2P	Point-to-Point	Wiring each system directly to each other.
SOA	Service-Oriented Architecture	Building software as reusable services.
TPS	Transaction Processing System	Front-line system recording day-to-day events.
UPT	Units Per Transaction	Units / transactions — items per visit.
CSV / XLSX	Comma-Sep. Values / Excel file	Plain-text and Excel spreadsheet formats.
SVG	Scalable Vector Graphics	Image format that stays crisp at any size.
OWASP	Open Worldwide App. Security Project	Where to learn to integrate systems securely.
CRA	Canada Revenue Agency	Canada's tax authority — a real external party.



Links worth bookmarking (1 / 2)

Enterprise software (Lesson 1)

- [SAP](#) — major ERP vendor
- [Oracle](#) — database & ERP/CRM
- [Microsoft Dynamics 365](#) — integrated ERP/CRM
- [Salesforce](#) — best-known cloud CRM

Standards & bodies (Lesson 1)

- [ANSI](#) — accredits US standards
- [ASC X12](#) — the dominant EDI format
- [ISO](#) — global standards body
- [GS1](#) — barcode & supply-chain IDs
- [W3C](#) — web standards (HTML, XML)
- [OASIS Open](#) — messaging / e-business specs



Links worth bookmarking (2 / 2)

Business Intelligence (Lesson 2)

- [Gartner BI glossary](#) — vendor-neutral definitions
- [Kimball Group](#) — the canon on data warehousing
- [Power BI](#) — widely used BI tool
- [Tableau](#) — leading visualisation
- [Qlik](#) — analytics & BI
- [Power Query](#) — mainstream ETL / data-prep
- [Talend](#) — data-integration / ETL

For students + safety

- [Python](#) — run every snippet; free
- [MDN Web Docs](#) — HTML/CSS/JS reference
- [Statistics Canada](#) — real open data to practise on
- [OWASP](#) — integrate systems securely
- [CRA](#) — a real external body
- [VirusTotal](#) — scan any link before clicking

Videos are linked, not hosted — see the copyright note on each lesson's video slide. Scan unfamiliar links with VirusTotal first.



Disclaimer, IP, Liability & Ethics (1 / 2)

- **Ownership & IP.** All original text, diagrams, datasets, and code are © Cyber.SoHo Educational Hub, all rights reserved. Use for personal study and classroom instruction; do not resell, rebrand, or redistribute for profit without written permission.
- **Original work — no infringement.** 'Northern Roast Coffee Co.' and all its figures are entirely fictional; any resemblance to a real company is coincidental. Real organizations are named descriptively, with links to official sources.
- **No affiliation.** Cyber.SoHo is an independent provider, not affiliated with, sponsored by, or endorsed by any company, vendor, platform, video creator, or institution named or linked here. Trademarks belong to their owners.
- **Third-party links & video.** External links and referenced videos are linked, not hosted — not copied, mirrored, or re-hosted. Linking is consistent with each platform's terms (e.g. YouTube ToS).



Disclaimer, IP, Liability & Ethics (2 / 2)

- **Verify links — security.** Before opening any external link, scan it with VirusTotal ([virustotal.com/gui/home/url](https://www.virustotal.com/gui/home/url)). Cyber.SoHo is not responsible for third-party sites or downloads.
- **Educational, 'as is', no warranty.** Provided for education only, with no warranties of any kind. The code is illustrative teaching code — not production-grade; review, test, and harden before any real use.
- **Limitation of liability.** To the fullest extent permitted by law, Cyber.SoHo is not liable for any damages arising from use of these materials or any linked resource.
- **Ethics & lawful use.** For lawful, ethical, defensive, educational use only. Unauthorized access or misuse is illegal under, e.g., Canada's Criminal Code s.342.1; US CFAA 18 U.S.C. § 1030; EU Directive 2013/40/EU; Spain's Ley Orgánica 10/1995. Always get authorization and handle data lawfully.



Thank you

Questions, and then we build.



Cyber.SoHo Educational Hub

From the Industry to the Classroom — Building the IT's Future, Today and Together!

© Cyber.SoHo — ALL Rights Reserved.